

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота №1

з дисципліни
«Алгоритми і структури даних»

Виконала:

студентка групи ІМ-44
Крошка Дар'я Олександрівна
номер у списку групи: 11

Перевірила:

Молчанова А. А.

Київ 2025

Постановка задачі:

1. Представити напрямлений та ненаправлений граfi із заданими параметрами так само, як у лабораторній роботі №3. Відмінність: коефіцієнт $k = 1.0 \cdot n^3 \cdot 0.01 \cdot n^4 \cdot 0.01 \cdot 0.3$. Отже, матриця суміжності A_{dir} направленого графа за варіантом формується таким чином:

- 1) встановлюється параметр (seed) генератора випадкових чисел, рівне номеру варіанту $n_1 n_2 n_3 n_4$;
- 2) матриця розміром $n \times n$ заповнюється згенерованими випадковими числами в діапазоні $[0, 2.0)$;
- 3) обчислюється коефіцієнт $k = 1.0 \cdot n^3 \cdot 0.01 \cdot n^4 \cdot 0.01 \cdot 0.3$, кожен елемент матриці множиться на коефіцієнт k ;
- 4) елементи матриці округлюються: 0 — якщо елемент менший за 1.0, 1 — якщо елемент більший або дорівнює 1.0. 2.

Обчислити:

- 1) степені вершин направленого і ненаправленого графів;
- 2) напівстепені виходу та заходу направленого графа;
- 3) чи є граф однорідним (регулярним), і якщо так, вказати степінь однорідності графа;
- 4) перелік висячих та ізольованих вершин. Результати вивести у графічне вікно, консоль або файл.

3. Змінити матрицю A_{dir} , коефіцієнт $k = 1.0 \cdot n^3 \cdot 0.005 \cdot n^4 \cdot 0.005 \cdot 0.27$.

4. Для нового орграфа обчислити:

- 1) півстепені вершин;
- 2) всі шляхи довжини 2 і 3;
- 3) матрицю досяжності;
- 4) матрицю сильної зв'язності;
- 5) перелік компонент сильної зв'язності;
- 6) граф конденсації. Результати вивести у графічне вікно, в консоль або файл. Шляхи довжиною 2 і 3 слід шукати за матрицями A_2 і A_3 , відповідно. Як результат вивести перелік шляхів, включно з усіма проміжними вершинами, через які проходить шлях. Матрицю досяжності та компоненти сильної зв'язності слід шукати за допомогою операції транзитивного замикання. У переліку компонент слід вказати, які вершини належать до кожної компоненти. Граф конденсації вивести у графічне вікно.

Текст програми:

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Лабораторна робота 4</title>
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    }

    body {
      background: linear-gradient(135deg, #E3F2FD, #BBDEFB);
      min-height: 100vh;
      display: flex;
      flex-direction: column;
      align-items: center;
      justify-content: center;
      padding: 20px;
    }

    h1 {
      color: #0D47A1;
      font-size: 2.4rem;
      font-weight: 700;
      margin-bottom: 30px;
      text-align: center;
      text-shadow: 1px 1px 6px rgba(0, 0, 0, 0.1);
      transition: color 0.3s ease-in-out;
    }

    h1:hover {
      color: #1E88E5;
```

```
}
```

```
.canvases {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 30px;  
  justify-content: center;  
  max-width: 100%;  
  margin-top: 20px;  
}
```

```
canvas {  
  background-color: #FFFFFF;  
  border: 4px solid #1565C0;  
  border-radius: 16px;  
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.1);  
  transition: transform 0.3s ease, box-shadow 0.3s ease;  
  cursor: pointer;  
}
```

```
canvas:hover {  
  transform: scale(1.05);  
  box-shadow: 0 12px 36px rgba(0, 0, 0, 0.2);  
}
```

```
#info {  
  margin-top: 20px;  
  padding: 20px;  
  background-color: rgba(255, 255, 255, 0.9);  
  border-radius: 12px;  
  border: 2px solid #BBDEFB;  
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);  
  max-width: 80%;  
  color: #0D47A1;  
  text-align: center;  
  display: none;  
}
```

```

    @media (max-width: 768px) {
        canvas {
            width: 90vw;
            height: 90vw;
        }
    }
</style>
</head>
<body>
    <h1>Лабораторна работа 4</h1>
    <div class="canvases">
        <canvas id="directedGraphCanvas" width="600" height="600"></canvas>
        <canvas id="undirectedGraphCanvas" width="600"
height="600"></canvas>
        <canvas id="condensedGraphCanvas" width="600"
height="600"></canvas>
    </div>
    <div id="info"></div>
    <script
src="https://cdn.jsdelivr.net/npm/seedrandom@3.0.5/seedrandom.min.js
"></script>
    <script>
        // The existing JavaScript code remains unchanged.
        const directedCanvas =
document.getElementById("directedGraphCanvas");
        const directedCtx = directedCanvas.getContext("2d");
        const undirectedCanvas =
document.getElementById("undirectedGraphCanvas");
        const undirectedCtx = undirectedCanvas.getContext("2d");
        const condensedCanvas =
document.getElementById("condensedGraphCanvas");
        const condensedCtx = condensedCanvas.getContext("2d");

        const n = 11;
        const radius = 20;
        const centerX = 300, centerY = 300, circleRadius = 220;

```

```

const seed = 4411;
const n3 = 1, n4 = 1;
let k = 1.0 - n3 * 0.01 - n4 * 0.01 - 0.3;

const nodePositions = Array.from({ length: n }, (_, i) => {
  const angle = (i * 2 * Math.PI) / n;
  return {
    x: centerX + circleRadius * Math.cos(angle),
    y: centerY + circleRadius * Math.sin(angle)
  };
});

function generateAdjacencyMatrix(directed) {
  let matrix = [];
  let rng = new Math.seedrandom(seed);
  for (let i = 0; i < n; i++) {
    matrix[i] = [];
    for (let j = 0; j < n; j++) {
      let value = rng() * 2.0 * k;
      matrix[i][j] = value >= 1.0 ? 1 : 0;
    }
  }
  if (!directed) {
    for (let i = 0; i < n; i++) {
      for (let j = i + 1; j < n; j++) {
        if (matrix[i][j] === 1) matrix[j][i] = 1;
      }
    }
  }
  return matrix;
}

function drawGraph(ctx, matrix, directed) {
  ctx.clearRect(0, 0, ctx.canvas.width, ctx.canvas.height);
  drawEdges(ctx, matrix, directed);
  drawNodes(ctx, nodePositions);
}

```

```

function drawNodes(ctx, positions) {
  positions.forEach((pos, idx) => {
    ctx.fillStyle = "#1565C0";
    ctx.beginPath();
    ctx.arc(pos.x, pos.y, radius, 0, Math.PI * 2);
    ctx.fill();
    ctx.strokeStyle = "#0D47A1";
    ctx.lineWidth = 3;
    ctx.stroke();

    ctx.fillStyle = "white";
    ctx.font = "16px Arial";
    ctx.textAlign = "center";
    ctx.textBaseline = "middle";
    ctx.fillText(idx + 1, pos.x, pos.y);
  });
}

function drawEdges(ctx, matrix, directed) {
  ctx.strokeStyle = "#0D47A1";
  ctx.lineWidth = 2;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      if (matrix[i][j] === 1) {
        const { x: x1, y: y1 } = nodePositions[i];
        const { x: x2, y: y2 } = nodePositions[j];
        const angle = Math.atan2(y2 - y1, x2 - x1);
        const x1_new = x1 + radius * Math.cos(angle);
        const y1_new = y1 + radius * Math.sin(angle);
        const x2_new = x2 - radius * Math.cos(angle);
        const y2_new = y2 - radius * Math.sin(angle);
        ctx.beginPath();
        ctx.moveTo(x1_new, y1_new);
        ctx.lineTo(x2_new, y2_new);
        ctx.stroke();
        if (directed) {

```

```

        drawArrow(ctx, x2_new, y2_new, angle);
    }
}
}
}
}

```

```

function drawArrow(ctx, x, y, angle) {
    const size = 10;
    ctx.save();
    ctx.translate(x, y);
    ctx.rotate(angle);
    ctx.fillStyle = "#0D47A1";
    ctx.beginPath();
    ctx.moveTo(0, 0);
    ctx.lineTo(-size, -size / 2);
    ctx.lineTo(-size, size / 2);
    ctx.closePath();
    ctx.fill();
    ctx.restore();
}

```

```

function transposeMatrix(matrix) {
    return matrix[0].map((_, colIndex) => matrix.map(row =>
row[colIndex]));
}

```

```

function kosaraju(matrix) {
    let visited = new Array(n).fill(false);
    let stack = [];
    let components = [];

```

```

    function dfs1(v) {
        visited[v] = true;
        for (let i = 0; i < n; i++) {
            if (matrix[v][i] === 1 && !visited[i]) {
                dfs1(i);
            }
        }
    }

```



```

    }
  }
  stack.push(v);
}

function dfs2(v, component, transposed) {
  visited[v] = true;
  component.push(v);
  for (let i = 0; i < n; i++) {
    if (transposed[v][i] === 1 && !visited[i]) {
      dfs2(i, component, transposed);
    }
  }
}

for (let i = 0; i < n; i++) {
  if (!visited[i]) dfs1(i);
}

let transposed = transposeMatrix(matrix);
visited.fill(false);

while (stack.length > 0) {
  let v = stack.pop();
  if (!visited[v]) {
    let component = [];
    dfs2(v, component, transposed);
    components.push(component);
  }
}
return components;
}

function buildCondensationGraph(components, componentMap, matrix) {
  const size = components.length;
  const condensed = Array.from({ length: size }, () => Array(size).fill(0));
  for (let i = 0; i < n; i++) {

```

```

for (let j = 0; j < n; j++) {
  if (matrix[i][j]) {
    const ci = componentMap[i];
    const cj = componentMap[j];
    if (ci !== cj) {
      condensed[ci][cj] = 1;
    }
  }
}
return condensed;
}

```

```

function drawCondensedGraph(ctx, condensedMatrix) {
  const size = condensedMatrix.length;
  const positions = Array.from({ length: size }, (_, i) => {
    const angle = (i * 2 * Math.PI) / size;
    return {
      x: centerX + circleRadius * Math.cos(angle),
      y: centerY + circleRadius * Math.sin(angle)
    };
  });
}

```

```

ctx.clearRect(0, 0, ctx.canvas.width, ctx.canvas.height);

```

```

condensedMatrix.forEach((row, i) => {
  row.forEach((val, j) => {
    if (val === 1) {
      const { x: x1, y: y1 } = positions[i];
      const { x: x2, y: y2 } = positions[j];
      const angle = Math.atan2(y2 - y1, x2 - x1);
      const x1_new = x1 + radius * Math.cos(angle);
      const y1_new = y1 + radius * Math.sin(angle);
      const x2_new = x2 - radius * Math.cos(angle);
      const y2_new = y2 - radius * Math.sin(angle);
      ctx.beginPath();
      ctx.moveTo(x1_new, y1_new);
    }
  });
}

```

```

        ctx.lineTo(x2_new, y2_new);
        ctx.stroke();
        drawArrow(ctx, x2_new, y2_new, angle);
    }
});
});

```

```

positions.forEach((pos, idx) => {
    ctx.fillStyle = "#1565C0";
    ctx.beginPath();
    ctx.arc(pos.x, pos.y, radius, 0, Math.PI * 2);
    ctx.fill();
    ctx.strokeStyle = "#0D47A1";
    ctx.lineWidth = 3;
    ctx.stroke();

    ctx.fillStyle = "white";
    ctx.font = "16px Arial";
    ctx.textAlign = "center";
    ctx.textBaseline = "middle";
    ctx.fillText(idx + 1, pos.x, pos.y);
});
}

```

```

const directedMatrix = generateAdjacencyMatrix(true);
const undirectedMatrix = generateAdjacencyMatrix(false);

```

```

drawGraph(directedCtx, directedMatrix, true);
drawGraph(undirectedCtx, undirectedMatrix, false);

```

```

const components = kosaraju(directedMatrix);

```

```

const componentMap = Array(n).fill(0);
components.forEach((component, idx) => {
    component.forEach(node => {
        componentMap[node] = idx;
    });
});

```

```

});

const condensedMatrix = buildCondensationGraph(components,
componentMap, directedMatrix);

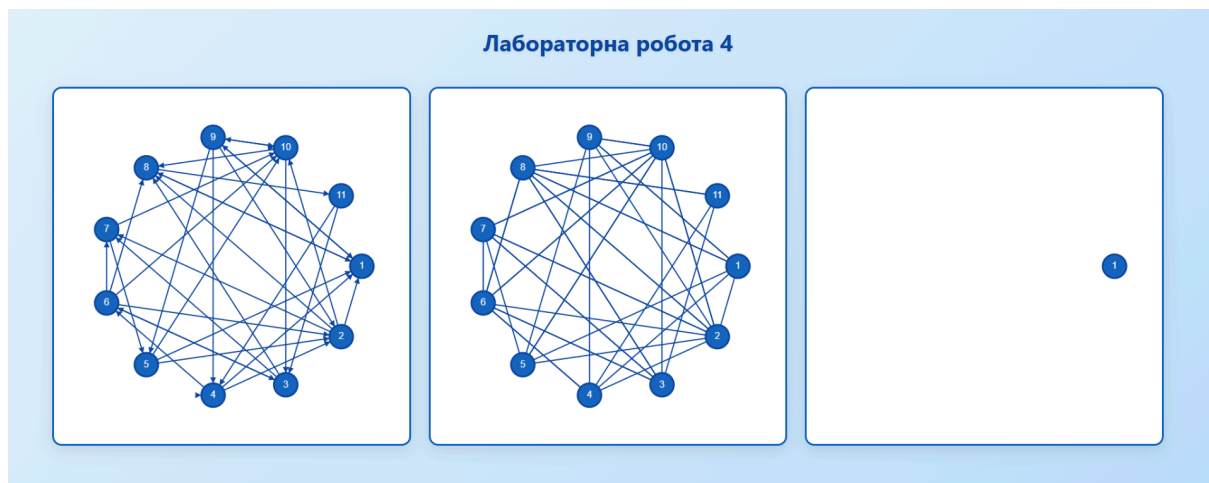
drawCondensedGraph(condensedCtx, condensedMatrix);

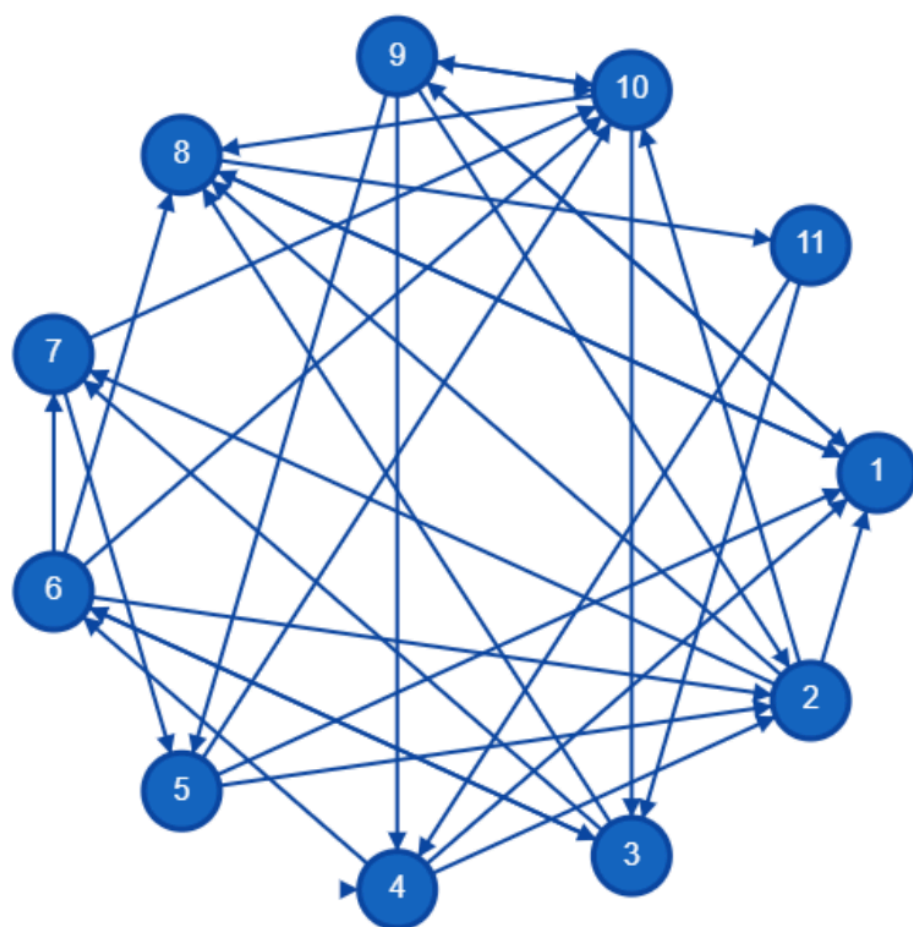
window.onload = () => {
  document.getElementById("info").textContent =
    `Компоненти сильної зв'язності:\n${components.map(c => c.map(i =>
i + 1)).join('\n')}`;
  console.log(components);
};

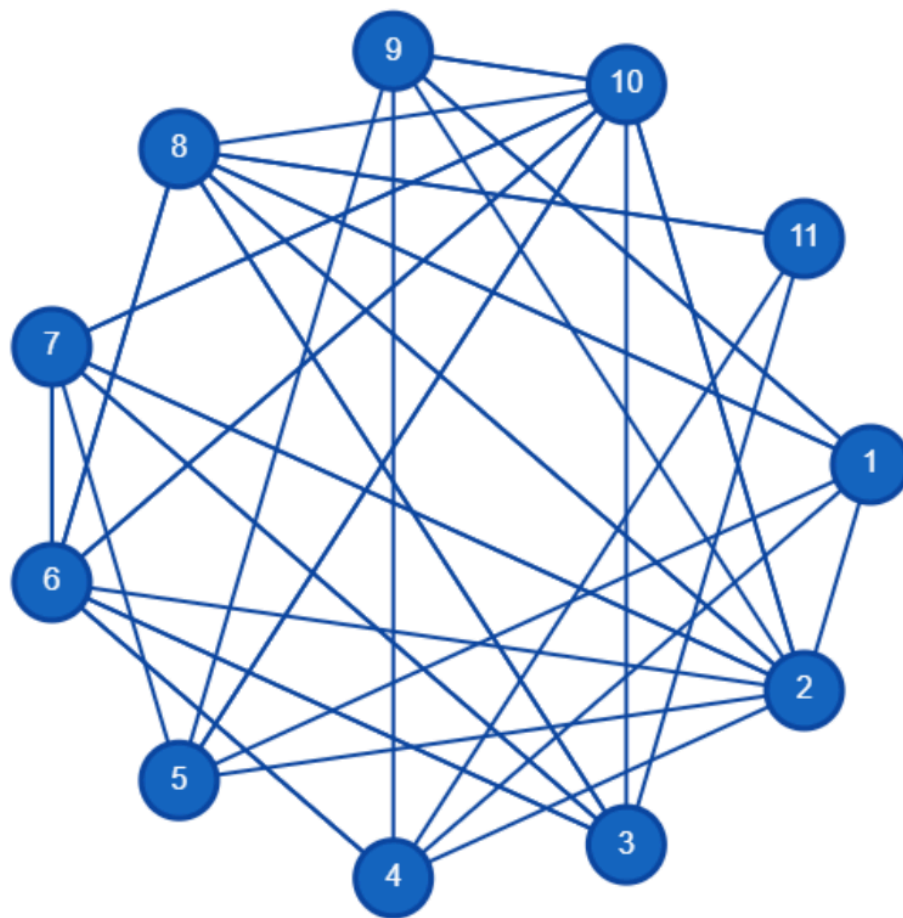
console.log("Кількість компонент:", components.length);
</script>
</body>
</html>

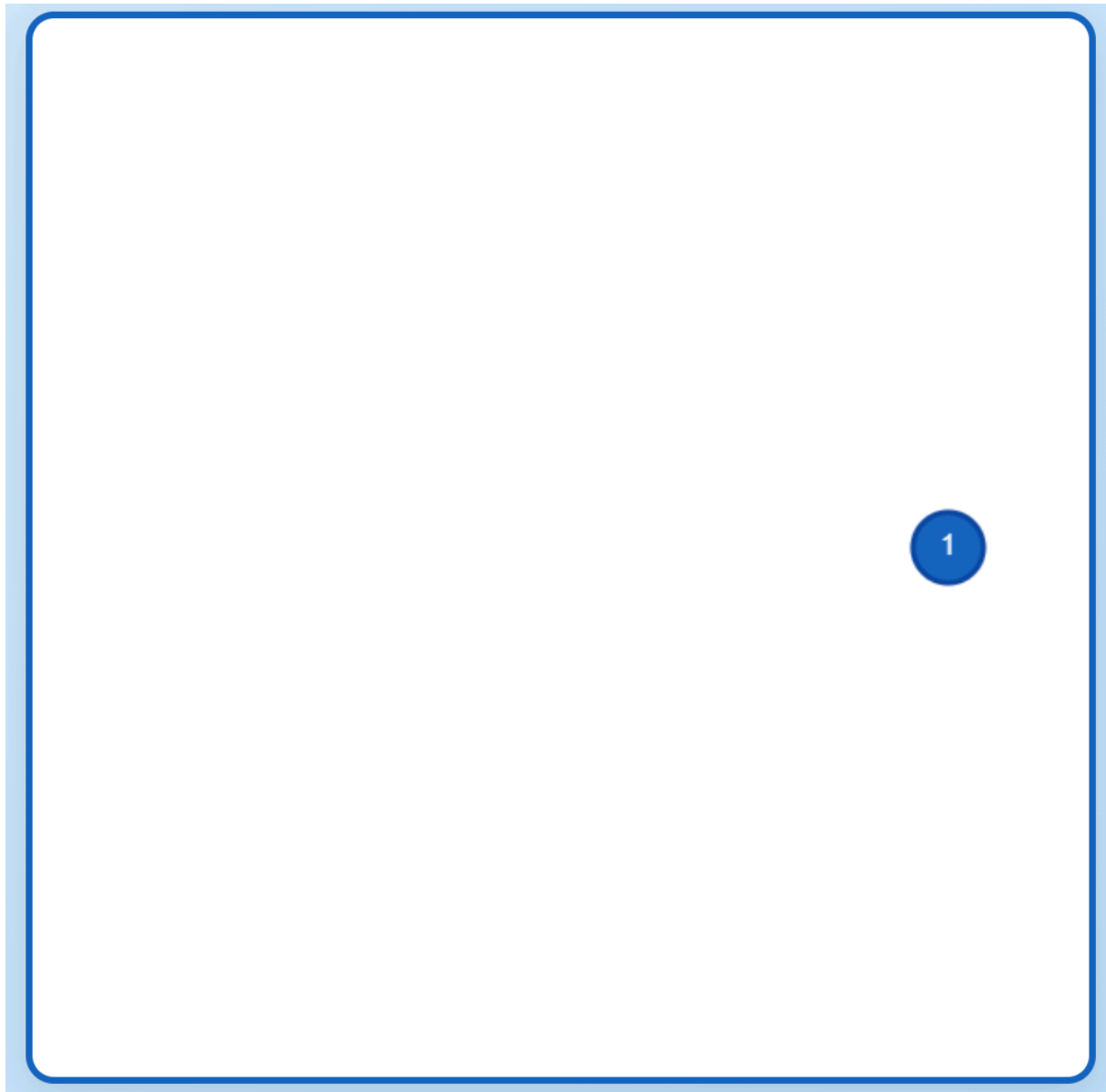
```

Тестування програми:









Компоненти сильної зв'язності: [1,2,4,9,10,5,7,3,6,11,8]

Висновки:

У рамках виконання лабораторної роботи були розглянуті напрямлені та ненаправлені графи, побудовані на основі випадкових значень, згенерованих за допомогою визначеного коефіцієнта k . Програма успішно реалізувала побудову матриць суміжності для обох типів графів, а також визначила важливі характеристики графів.

Результати були візуалізовані на графах, що дозволило детально вивчити структуру зв'язків у графах. Програма працює коректно і надає точні результати для всіх вказаних характеристик графів.

Загалом, лабораторна робота допомогла глибше зрозуміти методи аналізу графів та їх властивості, використовуючи алгоритми для пошуку компонент сильної зв'язності та матриць досяжності.